MHAI Phase 1 Frontend

Cursor Agent Build Guide — Complete Executable Specification

Step-by-step instructions to build the Phase 1 clinical frontend from the existing mockup + backend codebase

MHAI Project — 2026

How To Use This Document

This document is written to be handed directly to a **Cursor agent** (or Claude Code) together with two input files: (1) the MHAI backend codebase zip (`mhai-multiagent.zip`), and (2) the interactive HTML mockup (`mhai-clinical-mockup-v2.html`). Each "Day" below is a self-contained work session with explicit tasks, copy-paste-ready prompts, the exact files to create, and a Definition of Done. The agent should complete each day fully and verify the Definition of Done before proceeding. Work sequentially — later days depend on earlier ones.

1 Project Context (Read First)

1.1 What You Are Building

A **single-workflow clinical decision-support web application** for doctors, nurses, and clinicians. The core loop is: a clinician fills out a patient assessment form → submits it → watches a processing status screen → reviews a structured clinical result (diagnosis, trust level, evidence, citations) → agrees with it or suggests a correction.

1.2 Inputs You Have Been Given

Input File	What It Contains & How To Use It
`mhai-multiagent.zip`	The complete MHAI backend codebase. Use it to read the API endpoints (`src/mhai/` and the API layer), the Pydantic response schemas (`core/schemas.py`), and the decision logic. The frontend calls this backend. **Do not modify the backend.**
`mhai-clinical-mockup-v2.html`	The approved interactive design mockup. This is your **visual source of truth**. Every color, font, spacing value, animation, component, and interaction in the final app must match this mockup. Open it, read its CSS `:root` variables, and port them exactly.

1.3 The Golden Rule

Match the Mockup Exactly

The mockup `mhai-clinical-mockup-v2.html` has already been through design review and clinician-friendliness review. **Do not redesign it.** Your job is to convert it faithfully into a real, maintainable React application — not to improve the visuals. Port the exact color tokens, the IBM Plex fonts, the ECG animation, the trust badges, the plain-language safety checks, and the three-column layout. If you are ever unsure how something should look or behave, open the mockup and copy it.

1.4 Clinician-Friendly Language (Critical)

Never Show Technical Jargon to Clinicians

The end users are doctors and nurses, not engineers. The backend uses technical terms (trust score, NLI, semantic entropy, guardrail cascade, PASS/REGENERATE/ESCALATE). The UI must **translate all of these into plain clinical language**, exactly as the mockup does:

Backend term (never show)	Clinician-facing term (show this)
`trust_score: 0.73`	Confidence: High (with strength bars)
`PASS / REGENERATE / ESCALATE`	High confidence / Use with caution / Needs review
`NLI citation entailment`	Evidence-backed claims
`semantic entropy`	Answer stability / Consistent reasoning
`PII masking`	Protecting patient privacy
`guardrail cascade`	Safety checks
`KB#0001`	The actual guideline name (e.g. "GDM Guideline")

The translation mapping is fully demonstrated in the mockup — follow it precisely.

2 Scope (What To Build and What To Skip)

IN SCOPE — Build These	OUT OF SCOPE — Do NOT Build
✓ Clinical query form (7 fields)	✗ Login / authentication / RBAC
✓ Processing status screen	✗ Admin dashboard
✓ Clinical result view	✗ PDF upload / document ingestion UI
✓ Trust/confidence badge (plain language)	✗ Separate triage (QDA) screen
✓ Inline citations with popovers	✗ Standalone knowledge-base search
✓ Evidence & Safety panel	✗ Persistent / cross-session history
✓ Validate (I Agree) action	✗ Mobile / tablet-specific layouts
✓ Correct (Suggest Correction) form	✗ Custom charts / advanced visualizations
✓ 4 states: PASS/REGEN/ESCALATE/FAILED	✗ Custom branding / design system
✓ Session-only query list (sidebar)	✗ Hosting / cloud infrastructure
✓ Desktop, 1024px and above	✗ HIPAA / PHI frontend controls

Phase 1 Uses Test Data Only

Because Phase 1 runs on anonymized/synthetic data only, you do **not** need login, encryption, session timeouts, or any PHI-handling logic on the frontend. The backend still masks data internally — but the frontend has no security obligations for Phase 1.

3 Tech Stack and Required Resources

3.1 Technology Choices

Concern	Choice	Why
Framework	React 18 + Vite	Fast, simple SPA. (Next.js is fine too, but Vite is lighter for a single-workflow app with no SSR need.)
Language	TypeScript	Type safety against the API schemas.
Component library	shadcn/ui	Matches the clean clinical aesthetic; copy-paste components, no heavy dependency.
Styling	Tailwind CSS	shadcn/ui is built on it; port the mockup's tokens as Tailwind theme values.
Data fetching	TanStack Query (React Query)	Handles the submit-then-poll pattern cleanly with built-in retry/refetch.
Form handling	React Hook Form + Zod	Form state + validation mirroring the Pydantic rules.
Fonts	IBM Plex Sans + IBM Plex Mono	Exactly as in the mockup (via `@fontsource` or Google Fonts).
HTTP client	Native `fetch` wrapped in a typed client	No need for axios.
Testing	Vitest + React Testing Library	Fast, Vite-native unit/component testing.
Container	Docker + Nginx	For the final production static build.

3.2 Cursor Setup and Recommended Extensions

Tools & Extensions To Enable in Cursor

- **Node.js 20 LTS** — required runtime. Verify with `node --version`.
- **pnpm** (or npm) — package manager. `pnpm` is faster; either works.
- **Cursor / VS Code extensions**: ESLint, Prettier, Tailwind CSS IntelliSense, "Error Lens" (inline errors), and the built-in TypeScript language server.
- **Cursor Agent settings**: enable "Auto-run" for terminal commands during scaffolding, but review file writes. Point the agent at this document and the two input files as context.
- **Browser**: Chrome or Edge for testing (with React DevTools extension).
- **No paid plugins required.** Everything above is free/open-source.

3.3 Key npm Packages To Install

```
# Core
npm create vite@latest mhai-frontend -- --template react-ts
cd mhai-frontend
npm install

# Styling + component library
npm install -D tailwindcss postcss autoprefixer
npm install class-variance-authority clsx tailwind-merge lucide-react
npx shadcn@latest init          # initialize shadcn/ui

# Data + forms
npm install @tanstack/react-query
npm install react-hook-form zod @hookform/resolvers

# Fonts
npm install @fontsource/ibm-plex-sans @fontsource/ibm-plex-mono

# Testing (dev)
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom @testing-library/
    user-event
```

4 Backend API Contract (Extract From the Zip First)

Agent Task: Read the API Before Coding

Before writing any frontend code, unzip `mhai-multiagent.zip` and read these files to confirm the exact API contract:

- `src/mhai/core/schemas.py` — the Pydantic request/response models (field names, types, enums)
- The API route definitions (search for `@app.post`, `@router`, or the FastAPI app in `src/mhai/`)
- `src/mhai/agents/orchestrator.py` — to understand the decision values

If the running backend exposes Swagger, also open `http://<backend-url>/docs` to confirm live shapes. Build a typed `src/api/types.ts` that mirrors these schemas exactly.

4.1 Endpoints the Frontend Uses

Verb	Path	Purpose
POST	`/api/v1/queries`	Submit a clinical query. Returns `{query_id, status}`.
GET	`/api/v1/queries/{id}`	Poll status. Returns `{status, progress, current_stage}`.
GET	`/api/v1/queries/{id}/result`	Fetch the completed clinical result.
POST	`/api/v1/feedback`	Submit validate/correct feedback.
GET	`/healthz /readyz`	Health checks (used for a degraded-mode banner).

4.2 Result Object Shape (what the result view renders)

```
// GET /api/v1/queries/{id}/result  ->  ClinicalResultOut
{
  "query_id": "uuid",
  "diagnosis": "Gestational Diabetes Mellitus",
  "icd10_code": "O24.4",
  "response_text": "Based on the clinical presentation...",
  "citations": [
    { "chunk_id": "KB#0001", "claim_text": "...",
      "nli_label": "ENTAILMENT",        // ENTAILMENT | CONTRADICTION | NEUTRAL
      "nli_confidence": 0.92,
      "action": "PASS" }                // PASS | FLAG | REDACT
  ],
  "trust_score": 0.73,                  // 0.0 - 1.0
  "decision": "PASS",                   // PASS | REGENERATE | ESCALATE
  "guardrail_details": {
    "l1_persona":  { "drift": 0.03, "pass_threshold": true },
    "l2_entropy":  { "entropy": 0.42, "pass_threshold": true },
    "l3_nli":      { "pass_rate": 0.80, "claims_checked": 5, "model_loaded": true },
    "l4_trust":    { "trust_score": 0.73, "weights": {"alpha":0.35,...} },
    "cascade_pass": true
  },
  "retrieval_ms": 28.7,
  "generation_ms": 14200.0
}
```

4.3 Translating Backend Values To Clinician Language

`decision`	`trust_score`	UI Confidence Label + Color
`PASS`	≥ 0.55	"High" — green badge, 4–5 strength bars
`REGENERATE`	0.30--0.55	"Use with caution" — amber badge, 2–3 bars
`ESCALATE`	< 0.30	"Needs review" — red badge, 1–2 bars + caution banner

Citation Action Mapping

`action: "PASS"` → green "verified" tag with checkmark. `action: "FLAG"` → amber "needs review" tag. `action: "REDACT"` → red tag, struck through.

When `l3_nli.model_loaded == false`, show a small yellow notice on the Evidence panel: "Evidence checking is running in reduced-accuracy mode."

5 Mock Mode (Build Without a Live Backend)

Agent Task: Build a Mock Layer First

The backend may not be reachable during early development. Create a mock API layer that returns the four canonical example responses so the entire UI can be built and demoed offline. Control it with an env var `VITE_USE_MOCK=true`.

```
// src/api/mock/fixtures.ts  --- four canonical scenarios
export const MOCK_RESULTS = {
  pass:       { /* trust 0.73, decision PASS,  all citations verified   */ },
  regenerate: { /* trust 0.42, decision REGEN, one FLAG citation        */ },
  escalate:   { /* trust 0.22, decision ESCALATE, REDACT citation +banner*/ },
  failed:     { /* status: "failed", error: "Processing timed out"      */ },
};
```

Definition of Done — Mock Mode

With `VITE_USE_MOCK=true`, submitting the form cycles through the mock scenarios (or a scenario picker in dev) so every UI state can be viewed without the backend running.

6 Day-by-Day Build Plan

Each day is one focused work session (3–5 hours of active agent time). Complete the Definition of Done before moving on.

Day 1 — Project Scaffold + Design System

Goal: The app renders the empty three-column layout with the exact colors and fonts from the mockup.

6.0.1 Tasks

1. Scaffold the Vite + React + TypeScript project; install all packages from §2.3.
2. Initialize Tailwind + shadcn/ui. Import IBM Plex Sans/Mono.
3. Open the mockup, copy its CSS `:root` variables, and port them into `tailwind.config.ts` as theme colors (teal, navy, amber, red, green, ink, line families) and into a global stylesheet.
4. Build the app shell: a CSS grid with three columns — **Sidebar** (250px), **Center** (flexible), **Evidence Panel** (330px), collapsible.
5. Create empty placeholder components: `Sidebar`, `TopBar`, `CenterPanel`, `EvidencePanel`.

Copy-Paste Prompt for the Agent

```
``Read mhai-clinical-mockup-v2.html.  Extract every CSS custom property from its :root
block and map them into tailwind.config.ts as named theme colors, plus a globals.css
with the font-family and base styles.  Scaffold a Vite React-TS app with Tailwind and
shadcn/ui.  Build a three-column app shell (Sidebar 250px, Center flexible, Evidence
panel 330px, collapsible via a top-bar button) matching the mockup's layout proportions
and the dark-navy sidebar gradient.  Use IBM Plex Sans and IBM Plex Mono.  Do not add any
content yet --- just the empty styled shell.''
```

Definition of Done — Day 1

✓ `npm run dev` shows a three-column layout. ✓ Colors and fonts visually match the mockup.
✓ The evidence panel collapses/expands. ✓ No console errors.

Day 2 — Input Form + API Client

Goal: A clinician can fill out the patient form and submit it (to mock or real backend).

6.0.2 Tasks

1. Build the **Patient Assessment form** with fields: Age (number), Sex (select), Priority (select: routine/urgent/emergency), Symptoms (text), Clinical question (textarea, 10–2000 chars), Specialty area (optional select), and a collapsible "Additional context" with Medical history (textarea).
2. Wire validation with React Hook Form + Zod, mirroring the Pydantic constraints (question min 10 chars, age 0–120, required fields). Show inline error messages.
3. Build the typed API client (`src/api/client.ts`) and the types (`src/api/types.ts`) from §3. Include the mock switch from §4.
4. Set up TanStack Query. Create a `useSubmitQuery` mutation that POSTs the form and returns `query_id`.
5. Match the form's visual style to the mockup exactly (dark banner header, three-column top row, teal focus rings, "Analyze" submit button).

Copy-Paste Prompt for the Agent

```
``Build the Patient Assessment form component matching the form-card in the mockup
(dark banner, Age/Sex/Priority row, Symptoms, Clinical question textarea with char
hint, Specialty select, collapsible Additional context with Medical history).  Use
React Hook Form + Zod.  Validation:  age 0-120, clinical_question 10-2000 chars,
symptoms and question required.  Show inline errors.  Create src/api/types.ts
mirroring the ClinicalResultOut schema from the backend zip, and src/api/client.ts with
submitQuery/getStatus/getResult/submitFeedback functions plus a VITE_USE_MOCK switch.
Wire the form submit through a TanStack Query mutation.''
```

Definition of Done — Day 2

✓ Form renders and matches the mockup. ✓ Validation blocks empty/invalid submits with clear messages. ✓ Submitting returns a `query_id` (mock or real). ✓ API types compile with no `any`.

Day 3 — Processing View + Result View

Goal: The full form → processing → result flow works end to end.

6.0.3 Tasks

1. Build the **Processing view**: the ECG-pulse SVG animation (copy from mockup), the six-step vertical stepper with plain-language stage names ("Protecting patient privacy", "Searching medical guidelines", "Generating clinical assessment", "Verifying against evidence", "Running safety checks", "Finalizing recommendation"), and a progress bar with percentage.
2. Implement polling: a `useQueryStatus` hook using TanStack Query `refetchInterval` (~2s) that advances the stepper based on `current_stage`/`progress`, then fetches the result when `status == completed`.
3. Build the **Result view**: diagnosis heading, ICD-10 chip, confidence badge (High/Caution/Review with strength bars — map from `decision`/`trust_score` per §3.4), response text with inline citation tags, and the latency line ("Analysis completed in 14.4 seconds").

4. Manage the view state machine: `form` → `processing` → `result` (and back via "New Consultation").

Copy-Paste Prompt for the Agent

```
''Build the Processing view matching the mockup's proc-card:  the ECG SVG animation, a
six-step stepper with the plain-language stage labels, and a progress bar.  Implement
a polling hook with TanStack Query that hits GET /queries/{id} every 2s, advances the
stepper from current_stage, and on status==completed fetches GET /queries/{id}/result.
Then build the Result view matching result-card:  diagnosis title, ICD-10 chip,
confidence badge that maps decision+trust_score to High/Caution/Needs-review
with strength bars, response_text rendered with inline citation tags colored by
citation.action, and a latency line.  Drive the form->processing->result transition with
a view state machine.''
```

Definition of Done — Day 3

✓ Submitting shows the animated processing view. ✓ Stepper advances and transitions to the result. ✓ Result shows diagnosis, confidence badge, citations, latency — all matching the mockup. ✓ "New Consultation" returns to the form.

Day 4 — Evidence Panel + Feedback

Goal: The complete workflow including feedback submission works.

6.0.4 Tasks

1. Build the **Evidence & Safety panel** (right column) with three sections, all in plain language:
   - **Safety Checks**: a checklist derived from `guardrail_details` — "Patient data protected", "Consistent reasoning", "Evidence-backed claims (X of Y verified)", and warnings for any failed/flagged check.
   - **Confidence Breakdown**: labeled bars for "Evidence match", "Internal consistency", "Values & dosages", "Answer stability" (map from the four guardrail sub-scores — never show $\alpha/\beta/\gamma/\delta$ or "NLI").
   - **Sources Used**: cards with the guideline name, a "Verified"/"Needs Review" badge, the excerpt, and usage count.
2. Build the **Feedback actions**: "I Agree" button (POSTs `action:validate`) and "Suggest Correction" button that opens a form (corrected diagnosis + required reason, min 3 chars; POSTs `action:correct`).
3. Show confirmation banners on success; disable the buttons after submission.
4. Wire feedback through a `useSubmitFeedback` mutation.

Copy-Paste Prompt for the Agent

```
''Build the Evidence & Safety right panel matching the mockup:  (1) Safety Checks
checklist derived from guardrail_details in plain language, (2) Confidence Breakdown as
labeled bars mapping the guardrail sub-scores to Evidence match / Internal consistency
/ Values & dosages / Answer stability --- never show alpha/beta/gamma/delta or the
term NLI, (3) Sources Used cards with guideline name, Verified/Needs-Review badge,
excerpt, usage count.  Then build the feedback actions:  an I-Agree button and a
Suggest-Correction form (corrected diagnosis + required reason min 3 chars).  POST both
to /api/v1/feedback via a TanStack mutation, show a confirmation banner, and disable the
buttons after submit.''
```

Definition of Done — Day 4

✓ Evidence panel shows safety checks, confidence bars, and sources — all plain-language. ✓ "I Agree" and "Suggest Correction" both submit and confirm. ✓ Correction requires a reason. ✓ Buttons disable after submission.

Day 5 — States + Sidebar + Edge Cases

Goal: Every state a clinician could encounter is handled.

6.0.5 Tasks

1. Implement all **four result variants** driven by the API response:
   - **PASS** (High): green strip + badge, no banner.
   - **REGENERATE** (Caution): amber strip + badge, subtle caution note.
   - **ESCALATE** (Needs review): red strip + badge + prominent caution banner ("This result needs clinician review before any clinical action is taken").
   - **FAILED**: error card with a clear message and a "Return to form" / "Retry" action.
2. Build the **Sidebar** session query list: each submitted query appears with a plain-language confidence dot (High/Caution/Review) + a short title + relative time. Stored **in memory only** (no persistence). Clicking re-shows that result. "New Consultation" button clears the form.
3. Handle **edge cases**: request timeout, backend error/500, network failure — each with a friendly message and retry. Add the NLI fallback-mode notice when `model_loaded == false`. Add a degraded-mode banner if `/readyz` reports a component down.

Copy-Paste Prompt for the Agent

```
``Implement the four result states driven by decision + status:  PASS (green, no banner),
REGENERATE (amber, caution note), ESCALATE (red + prominent review-required banner, per
the mockup caution-banner), and FAILED (error card with Return-to-form/Retry).  Build
the in-memory session sidebar list (confidence dot + title + relative time, clicking
re-shows the result, New Consultation clears).  Add robust handling for timeout, 500, and
network errors with friendly messages and retry, plus the reduced-accuracy notice when
l3_nli.model_loaded is false and a degraded-mode banner from /readyz.''
```

Definition of Done — Day 5

✓ All four states render correctly and match the mockup. ✓ ESCALATE shows the review banner. ✓ Sidebar lists session queries and re-opens them. ✓ Timeout/error/network failures are handled gracefully.

Day 6 — Polish + Testing

Goal: Production-quality look; automated tests pass.

6.0.6 Tasks

1. Cross-browser check on **Chrome, Edge, Safari** at **1024px and above**. Fix any layout, spacing, or transition issues. Confirm no horizontal scroll at 1024px.
2. Polish micro-interactions: button hovers, focus rings, the fade-up animations, the ECG timing, badge transitions — match the mockup's feel.
3. Write **automated tests** (Vitest + RTL) for the critical logic:
   - Form validation (rejects short question, missing fields).

- Confidence mapping (`trust_score/decision` → correct label + color, at boundary values 0.30 and 0.55).
- Polling logic (advances stages, stops on completed/failed).
- Feedback submission (validate + correct paths; reason required).
- The four scenario renders (PASS/REGEN/ESCALATE/FAILED) via the mock fixtures.

4. Run ESLint + Prettier; fix all warnings. Ensure `tsc --noEmit` is clean.

Copy-Paste Prompt for the Agent

```
''Do a cross-browser and 1024px polish pass to match the mockup exactly (hovers, focus
rings, fade-up animations, ECG timing, badge transitions; no horizontal scroll at
1024px).  Then write Vitest + React Testing Library tests covering:  form validation,
the trust_score/decision -> confidence-label mapping at boundaries 0.30 and 0.55, the
polling state machine, feedback submission (validate and correct, reason required), and
the four scenario renders from the mock fixtures.  Fix all ESLint/Prettier/tsc issues.''
```

Definition of Done — Day 6

✓ Looks production-quality in Chrome/Edge/Safari at ≥1024px. ✓ All tests pass. ✓ `tsc`, ESLint, Prettier clean. ✓ No console errors/warnings.

Day 7 (if needed) — Docker + Handover

Goal: A deployable production build the client's team can host.

6.0.7 Tasks

1. Create a multi-stage **Dockerfile**: build stage (`npm run build`) → runtime stage (Nginx serving the static `dist/`).
2. Add an `nginx.conf` with SPA fallback (all routes → `index.html`) and a runtime env mechanism for the backend URL (`VITE_API_BASE_URL`).
3. Write the **README**: local dev (`npm run dev`), build (`npm run build`), Docker build/run, the env var for the backend URL, and how to toggle mock mode.
4. Produce the production build, run the container, and smoke-test the full workflow against the real backend (or mock if backend not ready).
5. Package deliverables: source repo + Dockerfile + README + test suite.

Copy-Paste Prompt for the Agent

```
''Create a multi-stage Dockerfile (Vite build -> Nginx static serve) with an nginx.conf
that has SPA fallback and reads the backend URL from VITE_API_BASE_URL at build/runtime.
Write a README covering local dev, build, Docker build/run, the backend-URL env var, and
the mock-mode toggle.  Produce the production build, run the container, and smoke-test
the full form->processing->result->feedback flow.''
```

Definition of Done — Day 7

✓ `docker build` + `docker run` serves the app. ✓ It talks to the backend via the env-configured URL. ✓ README is complete. ✓ Full workflow smoke-tested end to end.

7 Target Project Structure

Expected src/ Layout at Completion

```
mhai-frontend/
|-- src/
|   |-- api/
|   |   |-- client.ts            # submitQuery/getStatus/getResult/submitFeedback
|   |   |-- types.ts             # mirrors backend Pydantic schemas
|   |   |-- mock/fixtures.ts     # PASS/REGEN/ESCALATE/FAILED responses
|   |-- components/
|   |   |-- layout/              # Sidebar, TopBar, EvidencePanel, AppShell
|   |   |-- form/                # PatientForm + field components
|   |   |-- processing/          # EcgPulse, StageStepper, ProgressBar
|   |   |-- result/              # ResultCard, ConfidenceBadge, CitationTag, LatencyLine
|   |   |-- evidence/            # SafetyChecks, ConfidenceBars, SourceCards
|   |   |-- feedback/            # AgreeButton, CorrectionForm, ConfirmBanner
|   |   |-- ui/                  # shadcn/ui components
|   |-- hooks/                   # useSubmitQuery, useQueryStatus, useSubmitFeedback
|   |-- lib/                     # confidence mapping, formatters, cn()
|   |-- state/                   # view state machine, in-memory session store
|   |-- styles/                  # globals.css, tokens ported from mockup
|   |-- App.tsx, main.tsx
|-- tests/                       # Vitest + RTL specs
|-- Dockerfile, nginx.conf, README.md
|-- tailwind.config.ts, vite.config.ts, package.json
```

8 Final Acceptance Checklist

The build is complete when ALL of the following are true

- ✓ Clinician can submit the 7-field assessment form with validation
- ✓ Processing view animates through 6 plain-language stages with a progress bar
- ✓ Result shows diagnosis, ICD-10, confidence badge, citations, latency
- ✓ Confidence is shown as High / Use with caution / Needs review (never a raw number or "trust score")
- ✓ No technical jargon anywhere (no NLI, entropy, guardrail, PASS/REGEN/ESCALATE, KB#)
- ✓ Evidence panel shows plain-language safety checks, confidence bars, and named sources
- ✓ "I Agree" and "Suggest Correction" both submit feedback and confirm
- ✓ All four states render: PASS, REGENERATE, ESCALATE (with banner), FAILED
- ✓ Sidebar lists session queries and re-opens them
- ✓ Timeout / error / network failures handled gracefully
- ✓ Works in Chrome, Edge, Safari at 1024px and above
- ✓ Automated tests pass; TypeScript, ESLint, Prettier clean
- ✓ Dockerfile + Nginx build serves the app with env-configured backend URL
- ✓ README documents dev, build, Docker, env vars, and mock mode
- ✓ Visual fidelity to `mhai-clinical-mockup-v2.html` confirmed side by side

After the Build: Informal Clinician Validation

Once deployed, have 2–3 clinicians click through the app for ∼20 minutes each and give verbal feedback on: (1) is the form quick to fill, (2) is the result readable, (3) is the confidence level clear, (4) are the citations and safety checks understandable, (5) is Agree/Correct obvious. Fold the feedback into one short iteration. This replaces formal usability sessions for v1.

Estimated effort: 5–7 focused sessions · Cursor/Claude Code API cost: $100–300

Inputs required: mhai-multiagent.zip (backend) + mhai-clinical-mockup-v2.html (design source of truth)